NAME: KANIGASRI M

REG NO: 192565010

MySQL CODE

```
CREATE DATABASE employee_payroll;

USE employee_payroll;

CREATE TABLE department (
    department_id INT PRIMARY KEY AUTO_INCREMENT,
    department_name VARCHAR(50) NOT NULL UNIQUE
);

INSERT INTO department (department_name) VALUES
('IT'),
('HR'),
('Finance'),
('Marketing'),
('Sales');

CREATE TABLE employee (
    employee_id VARCHAR(10) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    gender VARCHAR(10),
    dob DATE,
    contact VARCHAR(15),
```

```
    email VARCHAR(100),
    address VARCHAR(200),
    department_id INT,
    designation VARCHAR(100),
    joining_date DATE,
    basic_salary DECIMAL(10,2) NOT NULL,
    allowances DECIMAL(10,2) DEFAULT 0,
    deductions DECIMAL(10,2) DEFAULT 0,
    employment_status VARCHAR(20),

    FOREIGN KEY (department_id)
    REFERENCES department(department_id)
);

INSERT INTO employee VALUES
('E101','Rohan Sharma','Male','1995-05-15',
'9876543210','rohan.sharma@email.com','New Delhi',
1,'Software Engineer','2022-06-01',
50000,5000,2000,'Active');

INSERT INTO employee VALUES
('E102','Priya Verma','Female','1994-10-10',
'9812345678','priya.verma@email.com','New Delhi',
2,'HR Executive','2021-03-15',
45000,4000,1500,'Active');

INSERT INTO employee VALUES
```

('E103','Amit Kumar','Male','1993-11-20',
'9711122334','amit.kumar@email.com','New Delhi',
3,'Accountant','2020-01-10',
48000,4500,1800,'Active');

---

2. JAVA CODE

.

```
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;
public class EmployeePayrollSystem extends JFrame {
   // Database details
   static final String URL =
         "jdbc:mysql://localhost:3306/employee_payroll";
   static final String USER = "root";
   static final String PASSWORD = "root";
   Connection con;
   EmployeePayrollSystem() {
      connectDatabase();
      setTitle("Employee Management & Payroll System");
      setSize(650, 600);
      setLayout(null);
      setLocationRelativeTo(null);
      Label title = new Label(
            "EMPLOYEE MANAGEMENT & PAYROLL SYSTEM",
            Label.CENTER);
```

```
title.setFont(new Font("Arial", Font.BOLD, 20));
title.setBounds(50, 50, 550, 40);
add(title);
Button register = new Button("Employee Registration");
register.setBounds(180, 120, 280, 45);
add(register);
Button search = new Button("Employee Search");
search.setBounds(180, 180, 280, 45);
add(search);
Button update = new Button("Employee Update");
update.setBounds(180, 240, 280, 45);
add(update);
Button delete = new Button("Employee Delete");
delete.setBounds(180, 300, 280, 45);
add(delete);
Button view = new Button("Employee List / View");
view.setBounds(180, 360, 280, 45);
add(view);
Button payroll = new Button("Payroll / Salary Calculation");
payroll.setBounds(180, 420, 280, 45);
add(payroll);
Button exit = new Button("Exit");
exit.setBounds(180, 480, 280, 45);
add(exit);
register.addActionListener(e -> registration());
```

```
    search.addActionListener(e -> searchEmployee());
    update.addActionListener(e -> updateEmployee());
    delete.addActionListener(e -> deleteEmployee());
    view.addActionListener(e -> viewEmployees());
    payroll.addActionListener(e -> payroll());
    exit.addActionListener(e -> System.exit(0));
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });
    setVisible(true);
}
// DATABASE CONNECTION
void connectDatabase() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(
                URL, USER, PASSWORD);
        System.out.println("Database connected successfully");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(
                this,
                "Database connection failed!\n"
                        + "Please check MySQL and password.",
                "Database Error",
                JOptionPane.ERROR_MESSAGE);
```

```
    }
}
// REGISTRATION
void registration() {
    Frame f = new Frame("Employee Registration");
    f.setSize(700, 700);
    f.setLayout(null);
    f.setLocationRelativeTo(this);
    Label heading = new Label(
            "EMPLOYEE REGISTRATION",
            Label.CENTER);
    heading.setFont(new Font("Arial", Font.BOLD, 20));
    heading.setBounds(150, 40, 400, 40);
    f.add(heading);
    Label l1 = new Label("Employee ID:");
    Label l2 = new Label("Name:");
    Label l3 = new Label("Gender:");
    Label l4 = new Label("Date of Birth:");
    Label l5 = new Label("Contact Number:");
    Label l6 = new Label("Email:");
    Label l7 = new Label("Address:");
    Label l8 = new Label("Department:");
    Label l9 = new Label("Designation:");
    Label l10 = new Label("Date of Joining:");
    Label l11 = new Label("Basic Salary:");
    Label l12 = new Label("Allowances:");
    Label l13 = new Label("Deductions:");
```

```
Label l14 = new Label("Employment Status:");
TextField id = new TextField();
TextField name = new TextField();
Choice gender = new Choice();
TextField dob = new TextField();
TextField contact = new TextField();
TextField email = new TextField();
TextArea address = new TextArea();
Choice department = new Choice();
TextField designation = new TextField();
TextField joining = new TextField();
TextField salary = new TextField();
TextField allowance = new TextField();
TextField deduction = new TextField();
Choice status = new Choice();
gender.add("Male");
gender.add("Female");
gender.add("Other");
department.add("IT");
department.add("HR");
department.add("Finance");
department.add("Marketing");
department.add("Sales");
status.add("Active");
status.add("Inactive");
int y = 100;
```

```
l1.setBounds(70, y, 130, 25);
id.setBounds(220, y, 350, 25);
l2.setBounds(70, y + 35, 130, 25);
name.setBounds(220, y + 35, 350, 25);
l3.setBounds(70, y + 70, 130, 25);
gender.setBounds(220, y + 70, 150, 25);
l4.setBounds(70, y + 105, 130, 25);
dob.setBounds(220, y + 105, 350, 25);
l5.setBounds(70, y + 140, 130, 25);
contact.setBounds(220, y + 140, 350, 25);
l6.setBounds(70, y + 175, 130, 25);
email.setBounds(220, y + 175, 350, 25);
l7.setBounds(70, y + 210, 130, 25);
address.setBounds(220, y + 210, 350, 50);
l8.setBounds(70, y + 270, 130, 25);
department.setBounds(220, y + 270, 150, 25);
l9.setBounds(70, y + 305, 130, 25);
designation.setBounds(220, y + 305, 350, 25);
l10.setBounds(70, y + 340, 130, 25);
joining.setBounds(220, y + 340, 350, 25);
l11.setBounds(70, y + 375, 130, 25);
salary.setBounds(220, y + 375, 350, 25);
l12.setBounds(70, y + 410, 130, 25);
allowance.setBounds(220, y + 410, 350, 25);
l13.setBounds(70, y + 445, 130, 25);
deduction.setBounds(220, y + 445, 350, 25);
```

```
l14.setBounds(70, y + 480, 130, 25);
status.setBounds(220, y + 480, 150, 25);
f.add(l1);
f.add(id);
f.add(l2);
f.add(name);
f.add(l3);
f.add(gender);
f.add(l4);
f.add(dob);
f.add(l5);
f.add(contact);
f.add(l6);
f.add(email);
f.add(l7);
f.add(address);
f.add(l8);
f.add(department);
f.add(l9);
f.add(designation);
f.add(l10);
f.add(joining);
f.add(l11);
f.add(salary);
f.add(l12);
f.add(allowance);
f.add(l13);
```

```
f.add(deduction);
f.add(l14);
f.add(status);
Button save = new Button("Save");
save.setBounds(230, 620, 100, 35);
Button clear = new Button("Clear");
clear.setBounds(350, 620, 100, 35);
f.add(save);
f.add(clear);
save.addActionListener(e -> {
   try {
      if (id.getText().trim().isEmpty()
            || name.getText().trim().isEmpty()
            || salary.getText().trim().isEmpty()) {
         JOptionPane.showMessageDialog(
               f,
               "Please fill all mandatory fields!",
               "Validation Error",
               JOptionPane.ERROR_MESSAGE);
         return;
      }
      double basic = Double.parseDouble(
            salary.getText());
      double allow = Double.parseDouble(
            allowance.getText().isEmpty()
                  ? "0"
                  : allowance.getText());
```

```
            double deduct = Double.parseDouble(
                    deduction.getText().isEmpty()
                            ? "0"
                            : deduction.getText());
            if (basic < 0 || allow < 0 || deduct < 0) {
               JOptionPane.showMessageDialog(
                        f,
                        "Salary values cannot be negative!");
               return;
            }
            String check =
                    "SELECT employee_id FROM employee WHERE
employee_id=?";
            PreparedStatement checkPs =
                    con.prepareStatement(check);
            checkPs.setString(1, id.getText());
            ResultSet rs = checkPs.executeQuery();
            if (rs.next()) {
               JOptionPane.showMessageDialog(
                        f,
                        "Duplicate Employee ID!",
                        "Error",
                        JOptionPane.ERROR_MESSAGE);
               return;
            }
            String sql =
                    "INSERT INTO employee VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)";
```

```
PreparedStatement ps =
        con.prepareStatement(sql);
ps.setString(1, id.getText());
ps.setString(2, name.getText());
ps.setString(3, gender.getSelectedItem());
ps.setDate(4, Date.valueOf(dob.getText()));
ps.setString(5, contact.getText());
ps.setString(6, email.getText());
ps.setString(7, address.getText());
int deptId =
        getDepartmentId(
                department.getSelectedItem());
ps.setInt(8, deptId);
ps.setString(9, designation.getText());
ps.setDate(10, Date.valueOf(joining.getText()));
ps.setDouble(11, basic);
ps.setDouble(12, allow);
ps.setDouble(13, deduct);
ps.setString(14, status.getSelectedItem());
ps.executeUpdate();
JOptionPane.showMessageDialog(
        f,
        "Employee registered successfully!",
        "Success",
        JOptionPane.INFORMATION_MESSAGE);
```

```
            f.dispose();
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(
                    f,
                    "Please enter valid numeric salary values!");
        } catch (IllegalArgumentException ex) {
            JOptionPane.showMessageDialog(
                    f,
                    "Date must be in YYYY-MM-DD format!");
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(
                    f,
                    "Database error: " + ex.getMessage());
        }
    });
    clear.addActionListener(e -> {
        id.setText("");
        name.setText("");
        dob.setText("");
        contact.setText("");
        email.setText("");
        address.setText("");
        designation.setText("");
        joining.setText("");
        salary.setText("");
        allowance.setText("");
        deduction.setText("");
```

```
        });
        f.setVisible(true);
    }
    // GET DEPARTMENT ID
    int getDepartmentId(String dept)
            throws SQLException {
        String sql =
                "SELECT department_id FROM department WHERE
department_name=?";
        PreparedStatement ps =
                con.prepareStatement(sql);
        ps.setString(1, dept);
        ResultSet rs =
                ps.executeQuery();
        if (rs.next()) {
            return rs.getInt(1);
        }
        return 1;
    }
    // SEARCH
    void searchEmployee() {
        Frame f = new Frame("Employee Search");
        f.setSize(650, 550);
        f.setLayout(null);
        f.setLocationRelativeTo(this);
        Label l1 = new Label("Search By:");
        Choice choice = new Choice();
```

```
choice.add("Employee ID");
choice.add("Department");
choice.add("Designation");
choice.add("Employment Status");
Label l2 = new Label("Search Text:");
TextField text = new TextField();
Button search = new Button("Search");
TextArea result = new TextArea();
l1.setBounds(50, 60, 100, 30);
choice.setBounds(160, 60, 180, 30);
l2.setBounds(50, 110, 100, 30);
text.setBounds(160, 110, 250, 30);
search.setBounds(430, 110, 100, 30);
result.setBounds(50, 170, 540, 300);
f.add(l1);
f.add(choice);
f.add(l2);
f.add(text);
f.add(search);
f.add(result);
search.addActionListener(e -> {
   try {
      String column = "employee_id";
      if (choice.getSelectedItem()
            .equals("Department")) {
         column = "d.department_name";
```

```
} else if (choice.getSelectedItem()
        .equals("Designation")) {
    column = "e.designation";
} else if (choice.getSelectedItem()
        .equals("Employment Status")) {
    column = "e.employment_status";
}
String sql =
        "SELECT e.*, d.department_name " +
        "FROM employee e " +
        "JOIN department d " +
        "ON e.department_id=d.department_id " +
        "WHERE " + column + " LIKE ?";
PreparedStatement ps =
        con.prepareStatement(sql);
ps.setString(
        1,
        "%" + text.getText() + "%");
ResultSet rs =
        ps.executeQuery();
result.setText("");
boolean found = false;
while (rs.next()) {
    found = true;
    result.append(
            "Employee ID: "
                    + rs.getString("employee_id")
```

```
                + "\n");
        result.append(
                "Name: "
                        + rs.getString("name")
                        + "\n");
        result.append(
                "Gender: "
                        + rs.getString("gender")
                        + "\n");
        result.append(
                "Department: "
                        + rs.getString("department_name")
                        + "\n");
        result.append(
                "Designation: "
                        + rs.getString("designation")
                        + "\n");
        result.append(
                "Basic Salary: "
                        + rs.getDouble("basic_salary")
                        + "\n");
        result.append(
                "Status: "
                        + rs.getString("employment_status")
                        + "\n");

        result.append(
```

```
                    "----------------------------\n");
            }
            if (!found) {
                result.setText(
                        "Employee not found!");
            }
        } catch (SQLException ex) {
            result.setText(
                    "Database error: "
                            + ex.getMessage());
        }
    });
    f.setVisible(true);
}
// VIEW ALL EMPLOYEES
void viewEmployees() {
    Frame f = new Frame("Employee List / View");
    f.setSize(900, 550);
    f.setLayout(null);
    f.setLocationRelativeTo(this);
    TextArea area = new TextArea();
    area.setBounds(30, 50, 830, 400);
    Button refresh = new Button("Refresh");
    refresh.setBounds(380, 470, 120, 35);
    f.add(area);
    f.add(refresh);
    refresh.addActionListener(e -> {
```

```
try {
    String sql =
            "SELECT e.employee_id,e.name,"
            + "d.department_name,e.designation,"
            + "e.basic_salary,e.employment_status "
            + "FROM employee e "
            + "JOIN department d "
            + "ON e.department_id=d.department_id";
    Statement st =
            con.createStatement();
    ResultSet rs =
            st.executeQuery(sql);
    area.setText(
            "ID\tName\tDepartment\tDesignation\tSalary\tStatus\n");
    area.append(
            "-------------------------------------------------------\n");
    while (rs.next()) {
        area.append(
                rs.getString("employee_id")
                        + "\t"
                        + rs.getString("name")
                        + "\t"
                        + rs.getString("department_name")
                        + "\t"
                        + rs.getString("designation")
                        + "\t"
```

```
                        + rs.getDouble("basic_salary")
                        + "\t"
                        + rs.getString("employment_status")
                        + "\n");
            }
        } catch (SQLException ex) {
            area.setText(
                    "Database error: "
                            + ex.getMessage());
        }
    });
    refresh.doClick();
    f.setVisible(true);
}
// UPDATE
void updateEmployee() {
    Frame f = new Frame("Employee Update");
    f.setSize(600, 500);
    f.setLayout(null);
    f.setLocationRelativeTo(this);
    Label l1 =
            new Label("Employee ID:");
    TextField id =
            new TextField();
    Label l2 =
            new Label("New Name:");
```

```
TextField name =
        new TextField();
Label l3 =
        new Label("Basic Salary:");
TextField salary =
        new TextField();
Label l4 =
        new Label("Allowances:");
TextField allowance =
        new TextField();
Label l5 =
        new Label("Deductions:");
TextField deduction =
        new TextField();
Button update =
        new Button("Update");
l1.setBounds(70, 70, 120, 30);
id.setBounds(200, 70, 280, 30);
l2.setBounds(70, 120, 120, 30);
name.setBounds(200, 120, 280, 30);
l3.setBounds(70, 170, 120, 30);
salary.setBounds(200, 170, 280, 30);
l4.setBounds(70, 220, 120, 30);
allowance.setBounds(200, 220, 280, 30);
l5.setBounds(70, 270, 120, 30);
deduction.setBounds(200, 270, 280, 30);
```

```
update.setBounds(230, 340, 120, 35);
f.add(l1);
f.add(id);
f.add(l2);
f.add(name);
f.add(l3);
f.add(salary);
f.add(l4);
f.add(allowance);
f.add(l5);
f.add(deduction);
f.add(update);
update.addActionListener(e -> {
   try {
      String check =
            "SELECT employee_id FROM employee "
                  + "WHERE employee_id=?";
      PreparedStatement cp =
            con.prepareStatement(check);
      cp.setString(1, id.getText());
      ResultSet rs =
            cp.executeQuery();
      if (!rs.next()) {
         JOptionPane.showMessageDialog(
               f,
               "Employee not found!");
```

```
    return;
}
if (name.getText().trim().isEmpty()) {
    JOptionPane.showMessageDialog(
            f,
            "Name cannot be empty!");
    return;
}
double basic =
        Double.parseDouble(
              salary.getText());
double allow =
        Double.parseDouble(
               allowance.getText());
double deduct =
        Double.parseDouble(
               deduction.getText());
String sql =
        "UPDATE employee SET "
               + "name=?, basic_salary=?, "
               + "allowances=?, deductions=? "
               + "WHERE employee_id=?";
PreparedStatement ps =
        con.prepareStatement(sql);
ps.setString(1, name.getText());
ps.setDouble(2, basic);
ps.setDouble(3, allow);
```

```
            ps.setDouble(4, deduct);
            ps.setString(5, id.getText());
            ps.executeUpdate();
            JOptionPane.showMessageDialog(
                    f,
                    "Employee details updated successfully!");
            f.dispose();
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(
                    f,
                    "Enter valid salary values!");
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(
                    f,
                    "Database error: "
                            + ex.getMessage());
        }
    });
    f.setVisible(true);
}
// DELETE
void deleteEmployee() {
    String id =
            JOptionPane.showInputDialog(
                    this,
                    "Enter Employee ID to delete:");
```

```
if (id == null || id.trim().isEmpty()) {
   return;
}
try {
   String check =
         "SELECT employee_id FROM employee "
               + "WHERE employee_id=?";
   PreparedStatement cp =
         con.prepareStatement(check);
   cp.setString(1, id);
   ResultSet rs =
         cp.executeQuery();
   if (!rs.next()) {
      JOptionPane.showMessageDialog(
            this,
            "Employee not found!");
      return;
   }
   int confirm =
         JOptionPane.showConfirmDialog(
               this,
               "Are you sure you want to delete "
                     + id + "?",
               "Confirm Delete",
               JOptionPane.YES_NO_OPTION);
   if (confirm == JOptionPane.YES_OPTION) {
```

```
            String sql =
                  "DELETE FROM employee "
                        + "WHERE employee_id=?";
            PreparedStatement ps =
                  con.prepareStatement(sql);
            ps.setString(1, id);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(
                  this,
                  "Employee deleted successfully!");
         }
      } catch (SQLException ex) {
         JOptionPane.showMessageDialog(
               this,
               "Database error: "
                     + ex.getMessage());
      }
   }
   // PAYROLL
   void payroll() {
      Frame f =
            new Frame(
                  "Payroll / Salary Calculation");
      f.setSize(500, 500);
      f.setLayout(null);
      f.setLocationRelativeTo(this);
```

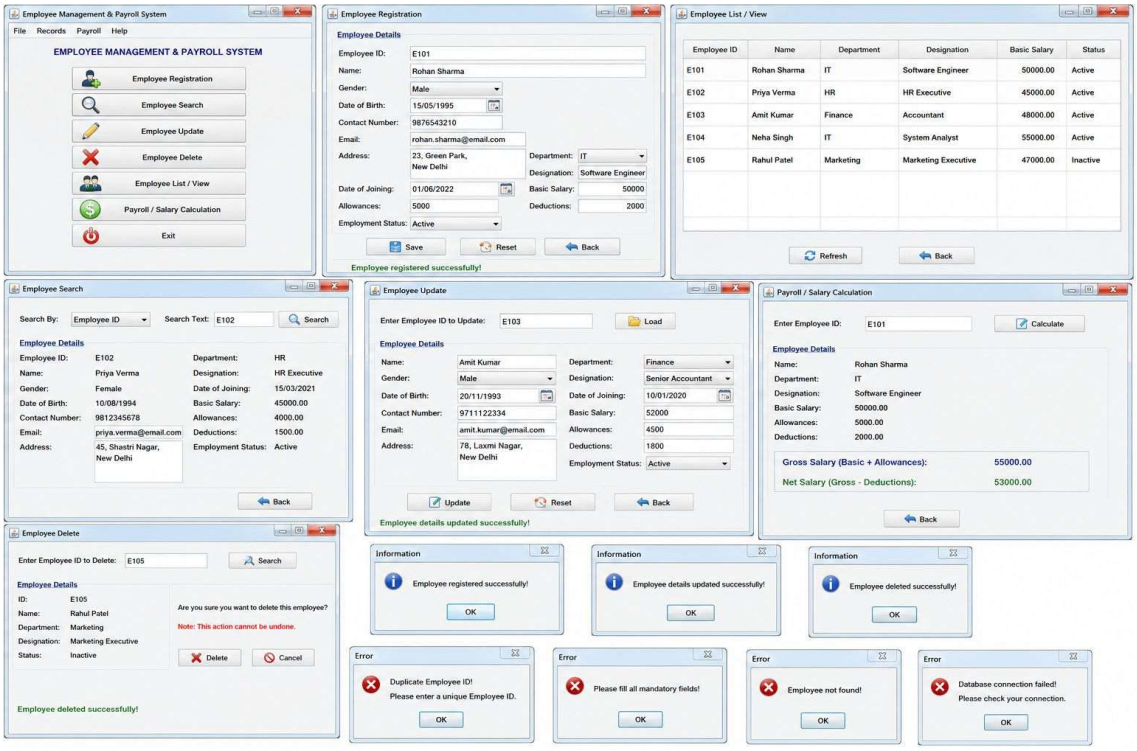
```
Label l =
        new Label("Enter Employee ID:");
TextField id =
        new TextField();
Button calculate =
        new Button("Calculate");
TextArea result =
        new TextArea();
l.setBounds(50, 60, 130, 30);
id.setBounds(200, 60, 150, 30);
calculate.setBounds(360, 60, 90, 30);
result.setBounds(50, 120, 400, 280);
f.add(l);
f.add(id);
f.add(calculate);
f.add(result);
calculate.addActionListener(e -> {
    try {
        String sql =
                "SELECT e.name,"
                        + "d.department_name,"
                        + "e.designation,"
                        + "e.basic_salary,"
                        + "e.allowances,"
                        + "e.deductions "
                        + "FROM employee e "
                        + "JOIN department d "
```

```
            + "ON e.department_id=d.department_id "
            + "WHERE e.employee_id=?";
PreparedStatement ps =
      con.prepareStatement(sql);
ps.setString(1, id.getText());
ResultSet rs =
      ps.executeQuery();
if (!rs.next()) {
   result.setText(
         "Employee not found!");
   return;
}
double basic =
      rs.getDouble("basic_salary");
double allowances =
      rs.getDouble("allowances");
double deductions =
      rs.getDouble("deductions");
double gross =
      basic + allowances;
double net =
      gross - deductions;
result.setText("");
result.append(
      "EMPLOYEE PAYROLL DETAILS\n");
result.append(
      "==============================\n\n");
```

```
result.append(
        "Employee Name: "
                + rs.getString("name")
                + "\n");
result.append(
        "Department: "
                + rs.getString("department_name")
                + "\n");
result.append(
        "Designation: "
                + rs.getString("designation")
                + "\n\n");
result.append(
        "Basic Salary: ₹"
                + basic
                + "\n");
result.append(
        "Allowances: ₹"
                + allowances
                + "\n");
result.append(
        "Deductions: ₹"
                + deductions
                + "\n\n");
result.append(
        "Gross Salary: ₹"
```

```
                        + gross
                        + "\n");

                result.append(
                        "Net Salary: ₹"
                        + net
                        + "\n");
            }
            catch (SQLException ex) {
                result.setText(
                        "Database error: "
                        + ex.getMessage());
            }
        });
        f.setVisible(true);
    }
    // MAIN METHOD
    public static void main(String[] args) {
        new EmployeePayrollSystem();
    }
}
```


Employee Management & Payroll System
File Records Payroll Help
EMPLOYEE MANAGEMENT & PAYROLL SYSTEM
Employee Registration
Employee Search
Employee Update
Employee Delete
Employee List / View
Payroll / Salary Calculation
Exit
Employee Registration
Employee Details
Employee ID: E101
Name: Rohan Sharma
Gender: Male
Date of Birth: 15/05/1995
Contact Number: 9876543210
Email: rohan.sharma@email.com
Address: 23, Green Park, New Delhi
Department: IT
Designation: Software Engineer
Date of Joining: 01/06/2022
Basic Salary: 50000
Allowances: 5000
Deductions: 2000
Employment Status: Active
Save
Reset
Back
Employee registered successfully!
Employee List / View
Employee ID | Name | Department | Designation | Basic Salary | Status
E101 | Rohan Sharma | IT | Software Engineer | 50000.00 | Active
E102 | Priya Verma | HR | HR Executive | 45000.00 | Active
E103 | Amit Kumar | Finance | Accountant | 48000.00 | Active
E104 | Neha Singh | IT | System Analyst | 55000.00 | Active
E105 | Rahul Patel | Marketing | Marketing Executive | 47000.00 | Inactive
Refresh
Back
Employee Search
Search By: Employee ID
Search Text: E102
Search
Employee Details
Employee ID: E102
Name: Priya Verma
Gender: Female
Date of Birth: 10/08/1994
Contact Number: 9812345678
Email: priya.verma@email.com
Address: 45, Shastri Nagar, New Delhi
Department: HR
Designation: HR Executive
Date of Joining: 15/03/2021
Basic Salary: 45000.00
Allowances: 4000.00
Deductions: 1500.00
Employment Status: Active
Back
Employee Update
Enter Employee ID to Update: E103
Load
Employee Details
Name: Amit Kumar
Gender: Male
Date of Birth: 20/11/1993
Contact Number: 9711122334
Email: amit.kumar@email.com
Address: 78, Laxmi Nagar, New Delhi
Department: Finance
Designation: Senior Accountant
Date of Joining: 10/01/2020
Basic Salary: 52000
Allowances: 4500
Deductions: 1800
Employment Status: Active
Update
Reset
Back
Employee details updated successfully!
Payroll / Salary Calculation
Enter Employee ID: E101
Calculate
Employee Details
Name: Rohan Sharma
Department: IT
Designation: Software Engineer
Basic Salary: 50000.00
Allowances: 5000.00
Deductions: 2000.00
Gross Salary (Basic + Allowances): 55000.00
Net Salary (Gross - Deductions): 53000.00
Back
Employee Delete
Enter Employee ID to Delete: E105
Search
Employee Details
ID: E105
Name: Rahul Patel
Department: Marketing
Designation: Marketing Executive
Status: Inactive
Are you sure you want to delete this employee?
Note: This action cannot be undone.
Delete
Cancel
Employee deleted successfully!
Information
Employee registered successfully!
OK
Information
Employee details updated successfully!
OK
Information
Employee deleted successfully!
OK
Error
Duplicate Employee ID!
Please enter a unique Employee ID.
OK
Error
Please fill all mandatory fields!
OK
Error
Employee not found!
OK
Error
Database connection failed!
Please check your connection.
OK